

Consistent Hashing

Understanding Hash Functions

- A hash function takes an input and maps it to a specific output.
- It is commonly used for distributing data efficiently in load balancing, caching, and sharding.

Normal Hashing (Modulo-Based Hashing)

How it works?

- Given N servers, we assign data using: $\text{ServerIndex} = \text{Hash}(\text{input}) \bmod N$
- Example: If $N = 3$ servers and we receive input X , we compute: $X \bmod 3$
- The result determines which of the 3 servers will handle the request.

Challenges of Modulo-Based Hashing:

- When a new server is added, we now mod by $(N+1)$ instead of N .
- This causes cache misses because:
 - Data previously mapped to a specific server is now mapped to a different one.
- Rebalancing is required to reassign data correctly.

Problem of Rebalancing

- Rebalancing means redistributing data when servers are added or removed.

Issue: With normal hashing, most of the data must be reassigned.

Solution: Consistent Hashing minimizes rebalancing.

Introduction to Consistent Hashing

- The goal of consistent hashing is to minimize the number of keys (data) that need to be reassigned when a server is added or removed.
- In a well-designed consistent hashing system, only $(1/N)$ of the total data needs to be reassigned when a server is added or removed.

Key Properties of Consistent Hashing

Minimal Rebalancing:

- Only $1/N$ of the total data needs reassignment when servers change.
- Formula for rebalancing: $1/N \times (\text{Total Number of Records})$, where N is the number of servers.

Works Well for Dynamic Scaling:

- Suitable when servers are frequently added or removed.
- Ideal for load balancing and database sharding.

How Consistent Hashing Works

Virtual Ring Structure

- Servers and data are arranged on a circular (virtual) hash ring.
- Example: A ring with 12 positions.
- Each server is mapped to specific points on the ring.

Data Assignment

- A request (or key) is mapped to a position on the ring.
- The request is assigned to the next available server in a clockwise direction.

Adding and Removing Servers

- When a new server is added, it takes over responsibility for a small portion of the ring.
- Only the affected portion of data is reassigned (instead of all data, as in normal hashing).

Virtual Nodes in Consistent Hashing

Issue: Uneven data distribution if servers are not evenly spaced.

Solution: Virtual Nodes

- Instead of placing one server at a single location, each server is mapped to multiple points on the ring.
- This ensures a balanced distribution of requests across servers.

Use Cases of Consistent Hashing

Load Balancing

- Distributes incoming requests evenly across multiple app servers.
- Minimizes disruption when scaling up or down.

Distributed Caching

- Used in Redis, Memcached to efficiently map data to cache nodes.
- Prevents frequent cache misses.

Database Sharding

- Helps in horizontal sharding by dynamically assigning data to different database partitions.